Fpt University

TRƯỜNG ĐẠI HỌC FPT

Android Service

Contents

- What is Service?
- Types of Services
- Service Lifecycle
- How to use Service
- Intent Service

What is Service?

- A Service is an application component that can perform long-running operations in the background and does not provide a user interface.
  - Started with an `Intent`.
  - Can stay running when user switches applications.
  - Lifecycle—which you must manage.
  - Other apps can use the service—manage permissions.
  - Runs in the main thread of its hosting process.

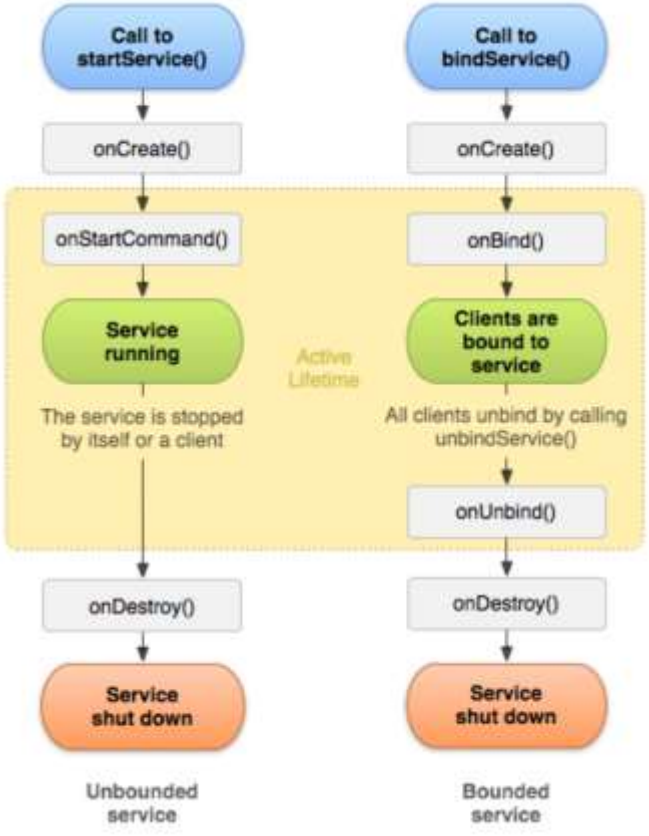
What is Service?

- **Service is good for:**
  - Network transactions.
  - Play music.
  - Perform file I/O.
  - Interact with a database.

Types of Services

- **Started Service**
  - Started with startService()
  - Runs indefinitely until it stops itself or called stopService() from outside
  - Usually does not update the UI
- **Bound Service**
  - Offers a client-server interface that allows components to interact with the service
  - Clients send requests and get results
  - Started with bindService()
  - Ends when all clients unbind

Service Lifecycle



- Life cycle
  - Service starts running when startService() / bindService() is called
  - Service is running even if called component is destroyed
  - It needs to be shutdown by stopService() / stopSelf()

Service Lifecycle

- **onCreate()**
  - Called when the service is being created (after first call of startService() or bindService())
- **onStartCommand()**
  - Called when startService() is called, delivers starting intent
  - Returned value specify behaviour when it’s killed by system
    - START_STICKY - don’t retain intent, later when system recreate service null intent is delivered (explicitly started/stopped services)
    - START_NOT_STICKY - if there is no start intent, take service out of the started state. Service is not recreated.
    - START_REDELIVER_INTENT - last delivered intent will be redelivered, pending intent delivered at the point of restart

Service Lifecycle

- **onBind()**
  - When another component binds to service
  - Returns Binder object for communication
- **onUnbind()**
  - When all clients disconnected from interface published by service
  - Returns true when onRebind should be called when new clients bind to service,

    otherwise onBind will be called

Service Lifecycle

- **onRebind()**
  - Called when new clients are connected, after notification about disconnecting all client

    in its onUnbind
- **onDestroy()**
  - Called by system to notify a Service that it is no longer used and is being removed.
  - Cleanup receivers, threads.

Foreground services

- Runs in the background but requires that the user is actively aware it exists—e.g. music player using music service
- Higher priority than background services since user will notice its absence—unlikely to be killed by the system
- Must provide a notification which the user cannot dismiss while the service is running

Background services limitations

- Starting from API 26, background app is not allowed to create a background service.
- A foreground app, can create and run both foreground and background services.
- When an app goes into the background, the system stops the app's background services.
- The startService() method now throws an IllegalStateException if an app is targeting API 26.
- These limitations don't affect foreground services or bound services.

Creating a service

- `<service android:name=".ExampleService" />`
- Manage permissions.
- Subclass `IntentService` or `Service` class.
- Implement lifecycle methods.
- Start service from `Activity`.
- Make sure service is stoppable.

Stopping a service

- A started service must manage its own lifecycle
- If not stopped, will keep running and consuming resources
- The service must stop itself by calling stopSelf()
- Another component can stop it by calling stopService()
- Bound service is destroyed when all clients unbound
- IntentService is destroyed after onHandleIntent() returns

Intent Service

- Subclass of Service
- Simple service with simplified lifecycle
- Uses worker thread to handle requests
- Handle only one request at one time
- Creates work queue
- Stops when it run out of work
- Override onHandleIntent(Intent) for processing requests, runs on worker thread

IntentService Limitations

- Cannot interact with the UI
- Can only run one request at a time
- Cannot be interrupted

IntentService Implementation

```
public class HelloIntentService extends IntentService {
    public HelloIntentService() { super("HelloIntentService");}

    @Override
  protected void onHandleIntent(Intent intent) {
      try {
          // Do some work
      } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
      }
  } // When this method returns, IntentService stops the service, as appropriate.
}
```

Classwork

- Create a play music service
- Start playing music in the background when we start the service and that music will play continuously until we stop the service in the android application.

References

- https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/